

Air University



DSA (Lab)

Year 2025

Mr. Ali Raza

Academic Year: 2024 – 2028
Department of Cyber Security

USAMA ARSHAD | 247141
BS – Cyber Security – Fall – 24 (A)
Date of Submission: December 02, 2025

Assignment # 03**Code**

```
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

struct Product
{
    int id;
    string name;
    int quantity;
    double price;
};

class Tnode
{
public:
    Product data;
    Tnode *left;
    Tnode *right;
};

// Binary Search Tree class for managing product inventory
class Tree
{
private:
    Tnode *root;

    // Creates a new tree node with the given product data
    Tnode *createNode(Product p)
    {
        Tnode *newNode = new Tnode();
        newNode->data = p;
        newNode->left = nullptr;
        newNode->right = nullptr;
        return newNode;
    }

    // Recursively inserts a product into the BST based on ID
    Tnode *insertNode(Tnode *node, Product p)
    {
        // If the current node is null, create a new node
        if (node == nullptr)
        {
            return createNode(p);
        }
        // If the product ID is less than current node's ID, insert into left subtree
        else if (p.id < node->data.id)
```

```
{
    node->left = insertNode(node->left, p);
}
// If the product ID is greater than current node's ID, insert into right subtree
else if (p.id > node->data.id)
{
    node->right = insertNode(node->right, p);
}
// If the product ID already exists, print error message
else
{
    cout << "Product Already Exists." << endl;
}
return node;
}

// Recursively searches for a product by ID in the BST
Tnode *searchNode(Tnode *node, int id)
{
    // If the current node is null, product not found
    if (node == nullptr)
    {
        return nullptr;
    }
    // If the ID matches, return the current node
    else if (id == node->data.id)
    {
        return node;
    }
    // If the ID is less than current node's ID, search left subtree
    else if (id < node->data.id)
    {
        return searchNode(node->left, id);
    }
    // Otherwise, search right subtree
    else
    {
        return searchNode(node->right, id);
    }
}

// Finds the in-order successor (smallest node in right subtree) for deletion
Tnode *findSuccessor(Tnode *node)
{
    // Traverse to the leftmost node in the subtree
    while (node != nullptr && node->left != nullptr)
    {
        node = node->left;
    }
    return node;
}
```

```
// Recursively deletes a product by ID from the BST
Tnode *deleteNode(Tnode *node, int id)
{
    // If the current node is null, nothing to delete
    if (node == nullptr)
    {
        return nullptr;
    }
    // If the ID is less than current node's ID, delete from left subtree
    else if (id < node->data.id)
    {
        node->left = deleteNode(node->left, id);
    }
    // If the ID is greater than current node's ID, delete from right subtree
    else if (id > node->data.id)
    {
        node->right = deleteNode(node->right, id);
    }
    // If the ID matches, handle deletion cases
    else
    {
        // Case 1: Node has no left child, return right child
        if (node->left == nullptr)
        {
            Tnode *temp = node->right;
            delete node;
            return temp;
        }
        // Case 2: Node has no right child, return left child
        else if (node->right == nullptr)
        {
            Tnode *temp = node->left;
            delete node;
            return temp;
        }
        // Case 3: Node has two children, replace with successor and delete successor
        else
        {
            Tnode *successor = findSuccessor(node->right);
            node->data = successor->data;
            node->right = deleteNode(node->right, successor->data.id);
        }
    }
    return node;
}

// Performs in-order traversal to display all products
void inOrder(Tnode *node)
{
    // Base case: if node is null, return
    if (node == nullptr)
    {
```

```
        return;
    }
    // Traverse left subtree
    inOrder(node->left);
    // Display current node's product details
    cout << "ID: " << node->data.id << endl;
    cout << "Name: " << node->data.name << endl;
    cout << "Quantity: " << node->data.quantity << endl;
    cout << "Price: " << fixed << setprecision(2) << node->data.price << endl;
    cout << "-----" << endl;
    // Traverse right subtree
    inOrder(node->right);
}

// Recursively counts total products and out-of-stock products
void countProducts(Tnode *node, int &total, int &outStock)
{
    // Base case: if node is null, return
    if (node == nullptr)
    {
        return;
    }
    // Increment total count
    total = total + 1;
    // If quantity is zero, increment out-of-stock count
    if (node->data.quantity == 0)
    {
        outStock = outStock + 1;
    }
    // Recurse on left and right subtrees
    countProducts(node->left, total, outStock);
    countProducts(node->right, total, outStock);
}

public:
    // Constructor initializes root to null
    Tree() : root(nullptr) {}

    // Public method to add a new product to the inventory
    void addProduct()
    {
        Product details;

        cout << "Enter Product ID: ";
        cin >> details.id;
        cin.ignore(); // Ignore newline character after ID input
        cout << "Enter Product Name: ";
        getline(cin, details.name);
        cout << "Enter Product Quantity: ";
        cin >> details.quantity;
        cout << "Enter Product Price: ";
        cin >> details.price;
```

```
// Insert the product into the BST
root = insertNode(root, details);
cout << "Product Added Successfully." << endl;
}

// Public method to search for a product by ID
void searchProduct()
{
    int id;
    cout << "Enter Product ID: ";
    cin >> id;
    // Search for the product
    Tnode *foundProduct = searchNode(root, id);
    // If not found, display message
    if (foundProduct == nullptr)
    {
        cout << "Product does not Exist." << endl;
    }
    // If found, display product details
    else
    {
        cout << "Product ID: " << foundProduct->data.id << endl;
        cout << "Product Name: " << foundProduct->data.name << endl;
        cout << "Product Quantity: " << foundProduct->data.quantity << endl;
        cout << "Product Price: " << fixed << setprecision(2) << foundProduct-
>data.price << endl;
    }
}

// Public method to delete a product by ID
void deleteProduct()
{
    int id;
    cout << "Enter Product ID: ";
    cin >> id;

    // Check if product exists
    Tnode *foundProduct = searchNode(root, id);

    // If not found, display message
    if (foundProduct == nullptr)
    {
        cout << "Product does not exist." << endl;
    }
    // If found, delete it
    else
    {
        root = deleteNode(root, id);
        cout << "Product Deleted Successfully." << endl;
    }
}
```

```
// Public method to display the entire inventory in in-order
void traverseInorder()
{
    cout << "===Inventory===" << endl;
    inOrder(root);
}

// Public method to update the quantity of a product
void updateQuantity()
{
    int id, newQuantity;

    cout << "Enter Product ID: ";
    cin >> id;

    // Search for the product
    Tnode *node = searchNode(root, id);

    // If not found, display message
    if (node == nullptr)
    {
        cout << "Product " << id << " Not Found." << endl;
    }
    // If found, update quantity
    else
    {
        cout << "Enter New Quantity: " << endl;
        cin >> newQuantity;

        node->data.quantity = newQuantity;
        cout << "Quantity Updated Successfully." << endl;
    }
}

// Public method to display total products and out-of-stock counts
void displayCounts()
{
    int total = 0;
    int outStock = 0;
    // Count products recursively
    countProducts(root, total, outStock);
    cout << "Total Products: " << total << endl;
    cout << "Out of Stock: " << outStock << endl;
}

};

// Main function to run the inventory management system
int main()
{
    Tree inventory;
    int choice;
```

```
// Infinite loop for menu-driven interface
while (true)
{
    cout << "\n=== Product Inventory Management ===" << endl;
    cout << "1. Add Product" << endl;
    cout << "2. Search Product" << endl;
    cout << "3. Delete Product" << endl;
    cout << "4. Update Quantity" << endl;
    cout << "5. Display Inventory" << endl;
    cout << "6. Display Counts (Total and Out of Stock)" << endl;
    cout << "7. Exit" << endl;
    cout << "Enter your choice: ";
    cin >> choice;

    // Switch based on user choice
    switch (choice)
    {
        case 1: // Add a new product
            inventory.addProduct();
            break;
        case 2: // Search for a product
            inventory.searchProduct();
            break;
        case 3: // Delete a product
            inventory.deleteProduct();
            break;
        case 4: // Update product quantity
            inventory.updateQuantity();
            break;
        case 5: // Display all products
            inventory.traverseInorder();
            break;
        case 6: // Display counts
            inventory.displayCounts();
            break;
        case 7: // Exit the program
            cout << "Exiting..." << endl;
            return 0;
        default: // Invalid choice
            cout << "Invalid choice. Please try again." << endl;
    }
}

return 0;
}
```


Output

```
=== Product Inventory Management ===
1. Add Product
2. Search Product
3. Delete Product
4. Update Quantity
5. Display Inventory
6. Display Counts (Total and Out of Stock)
7. Exit
Enter your choice: 1
Enter Product ID: 2001
Enter Product Name: Books
Enter Product Quantity: 15
Enter Product Price: 20.75
Product Added Successfully.
```

```
=== Product Inventory Management ===
1. Add Product
2. Search Product
3. Delete Product
4. Update Quantity
5. Display Inventory
6. Display Counts (Total and Out of Stock)
7. Exit
Enter your choice: 1
Enter Product ID: 2002
Enter Product Name: Pencils
Enter Product Quantity: 24
Enter Product Price: 5.50
Product Added Successfully.
```

```
=== Product Inventory Management ===
1. Add Product
2. Search Product
3. Delete Product
4. Update Quantity
5. Display Inventory
6. Display Counts (Total and Out of Stock)
7. Exit
Enter your choice: 1
Enter Product ID: 2003
Enter Product Name: Highlighter
Enter Product Quantity: 12
Enter Product Price: 7.50
Product Added Successfully.
```

```
=== Product Inventory Management ===
1. Add Product
2. Search Product
3. Delete Product
4. Update Quantity
5. Display Inventory
6. Display Counts (Total and Out of Stock)
7. Exit
Enter your choice: 2
Enter Product ID: 2002
Product ID: 2002
Product Name: Pencils
Product Quantity: 24
Product Price: 5.50
```

```
=== Product Inventory Management ===
1. Add Product
2. Search Product
3. Delete Product
4. Update Quantity
5. Display Inventory
6. Display Counts (Total and Out of Stock)
7. Exit
Enter your choice: 3
Enter Product ID: 2002
Product Deleted Successfully.
```

```
=== Product Inventory Management ===
1. Add Product
2. Search Product
3. Delete Product
4. Update Quantity
5. Display Inventory
6. Display Counts (Total and Out of Stock)
7. Exit
Enter your choice: 4
Enter Product ID: 2003
Enter New Quantity:
18
Quantity Updated Successfully.
```

```
=== Product Inventory Management ===
1. Add Product
2. Search Product
3. Delete Product
4. Update Quantity
5. Display Inventory
6. Display Counts (Total and Out of Stock)
7. Exit
Enter your choice: 5
===Inventory===
ID: 2001
Name: Books
Quantity: 15
Price: 20.75
-----
ID: 2003
Name: Highlighter
Quantity: 18
Price: 7.50
-----
```

```
=== Product Inventory Management ===
1. Add Product
2. Search Product
3. Delete Product
4. Update Quantity
5. Display Inventory
6. Display Counts (Total and Out of Stock)
7. Exit
Enter your choice: 6
Total Products: 2
Out of Stock: 0
```

```
=== Product Inventory Management ===
1. Add Product
2. Search Product
3. Delete Product
4. Update Quantity
5. Display Inventory
6. Display Counts (Total and Out of Stock)
7. Exit
Enter your choice: 8
Invalid choice. Please try again.
```

```
=== Product Inventory Management ===
1. Add Product
2. Search Product
3. Delete Product
4. Update Quantity
5. Display Inventory
6. Display Counts (Total and Out of Stock)
7. Exit
Enter your choice: 1
Enter Product ID: 2002
Enter Product Name: Pen
Enter Product Quantity: 24
Enter Product Price: 5.50
Product Added Successfully.
```

```
=== Product Inventory Management ===
1. Add Product
2. Search Product
3. Delete Product
4. Update Quantity
5. Display Inventory
6. Display Counts (Total and Out of Stock)
7. Exit
Enter your choice: 5
===Inventory===
ID: 2001
Name: Books
Quantity: 15
Price: 20.75
-----
ID: 2002
Name: Pen
Quantity: 24
Price: 5.50
-----
ID: 2003
Name: Highlighter
Quantity: 18
Price: 7.50
-----
```

```
=== Product Inventory Management ===
1. Add Product
3. Delete Product
4. Update Quantity
5. Display Inventory
6. Display Counts (Total and Out of Stock)
7. Exit
Enter your choice: 7
Exiting...
```

THE END
